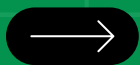


SUMMER COURSE WEEK 5



DUE TO 15.08.25 @ 10:00 AM

GOOD LUCK! :)

משימה 1

במשימה הזו תתבקשו לעבוד שלושתכם ביחד. החשיבות במשימה זו היא חלוקת העבודה בינכם, ולכן דבר ראשון שתגישו לנו (עוד לפני תחילת כתיבת הפרויקט) הוא מסמך word המתאר את החלוקה בניכם ותיאור המודולים של הפרויקט. כל אחד יהיה אחראי על חלק נפרד בפרויקט ולמעשה תיהיו תלויים אחד בשני. לאחר שנאשר את המסמך תוכלו להתחיל לעבוד על כתיבת הפרויקט עצמו. אנו מצפים לרמה גבוהה ולסטנדרט גבוה ולראות את העבודה בצוות שלכם והעזרה אחד לשני אז קחו זאת בחשבון. בנוסף, נעקוב אחרי העבודה שלכם ב-git נפתח לכם repo נפרד ונרצה לראות אתכם פותרים קונפליקטים, וסנכרון של סטנדרט צוותי וכל שאר הדברים עליהם דיברנו. במידה ויש בעיה כלשהי נא לפנות לאחד המנחים.

עליכם לכתוב תוכנת אסמבלר, עבור שפת אסמבלי שתוגדר בהמשך. הפרויקט ייכתב בשפת C.

עליכם להגיש את הפריטים הבאים :

1. קבצי המקור של התוכנית שכתבתם (קבצים בעלי סיומת c או h).
 2. קובץ הרצה (מקומפל ומקושר) עבור מערכת אובונטו.
 3. קובץ makefile. הקימפול חייב להיות עם הקומפיילר gcc והדגלים : -Wall -ansi -pedantic. יש לנפות את כל ההודעות שמוציא הקומפיילר, כך שהתוכנית תתקמפל ללא כל הערות או אזהרות.
 4. דוגמאות הרצה (קלט ופלט) :
- א.** קבצי קלט בשפת אסמבלי, וקבצי הפלט שנוצרו מהפעלת האסמבלר על קבצי קלט אלה. יש להדגים שימוש במגוון הפעולות וטיפוסי הנתונים של שפת האסמבלי.
- ב.** קבצי קלט בשפת אסמבלי המדגימים מגוון רחב של סוגי שגיאות אסמבלי (ולכן לא נוצרים קבצי פלט), ותדפיסי המסך המראים את הודעות השגיאה שמוציא האסמבלר.

בשל גודל הפרויקט, עליכם לחלק את התוכנית למספר קבצי מקור, לפי משימות. יש להקפיד שקוד המקור של התוכנית יעמוד בקריטריונים של בהירות, קריאות וכתובה נאה ומובנית.

נזכיר מספר היבטים חשובים של כתיבת קוד טוב :

1. הפשטה של מבני הנתונים : רצוי (ככל האפשר) להפריד בין הגישת למבני הנתונים לבין המימוש של מבני הנתונים. כך, למשל, בעת כתיבת פונקציות לטיפול בטבלה, אין זה מעניינם של המשתמשים בפונקציות אלה, האם הטבלה ממומשת באמצעות מערך או באמצעות רשימה מקושרת.
2. קריאות הקוד : יש להשתמש בשמות משמעותיים למשתנים ופונקציות. יש לערוך את הקוד באופן מסודר : הזחות עקביות, שורות ריקות להפרדה בין קטעי קוד, וכד'.
3. תיעוד : יש להכניס בקבצי המקור תיעוד תמציתי וברור, שיסביר את תפקידה של כל פונקציה (באמצעות הערות כותרת לכל פונקציה). כמו כן יש להסביר את תפקידם של משתנים חשובים. כמו כן, יש להכניס הערות ברמת פירוט טובה בכל הקוד.

הערה: תוכנית "עובדת", דהיינו תוכנית שמבצעת את כל הדרוש ממנה, אינה לכשעצמה ערובה לציון גבוה. כדי לקבל ציון גבוה, על התוכנית לעמוד בקריטריונים של כתיבה ותייעוד ברמה טובה, כמותאור לעיל, אשר משקלם המשותף מגיע עד לכ- 40% ממשקל הפרויקט.

מותר להשתמש בפרויקט בכל מגוון הספריות הסטנדרטיות של שפת C, אבל אין להשתמש בספריות חיצוניות אחרות.

מומלץ לעבוד בזוגות. אין לעבוד בצוותים גדולים יותר. **פרויקט שיוגש על ידי שלשה או יותר, לא ייבדק ולא יקבל ציון.** חובה שסטודנטים, הבוחרים להגיש יחד את הפרויקט, יהיו **שייכים לאותה קבוצת הנחיה.** הציון יהיה זהה לשני הסטודנטים.

מומלץ לקרוא את הגדרת הפרויקט פעם ראשונה ברצף, לקבלת תמונה כללית לגבי הנדרש, ורק לאחר מכן לקרוא שוב בצורה מעמיקה יותר.

רקע כללי ומטרת הפרויקט

כידוע, קיימות שפות תכנות רבות, ומספר גדול של תוכניות, הכתובות בשפות שונות, עשויות לרוץ באותו מחשב עצמו. כיצד "מכיר" המחשב כל כך הרבה שפות? התשובה פשוטה: המחשב מכיר למעשה שפה אחת בלבד: הוראות ונתונים הכתובים בקוד בינארי. קוד זה מאוחסן בגוש בזיכרון, ונראה כמו רצף של ספרות בינאריות. יחידת העיבוד המרכזית - היע"מ (CPU) - יודעת לפרק את הרצף הזה לקטעים קטנים בעלי משמעות: הוראות, מענים ונתונים.

למעשה, זיכרון המחשב כולו הוא אוסף של סיביות, שנוהגים לראותן כמקובצות ליחידות בעלות אורך קבוע (בתים, מילים). לא ניתן להבחין, בעין שאינה מיומנת, בהבדל פיסי כלשהו בין אותו חלק בזיכרון שבו נמצאת תוכנית לבין שאר הזיכרון.

יחידת העיבוד המרכזית (היע"מ) יכולה לבצע מגוון פעולות פשוטות, הנקראות **הוראות מכונה**, ולשם כך היא משתמשת באוגרים (registers) הקיימים בתוך היע"מ, ובזיכרון המחשב. **דוגמאות:** העברת מספר מתא בזיכרון לאוגר ביע"מ או בחזרה, הוספת 1 למספר הנמצא באוגר, בדיקה האם מספר המאוחסן באוגר שווה לאפס, חיבור וחסור בין שני אוגרים, וכד'. הוראות המכונה ושילובים שלהן הן המרכיבות תוכנית כפי שהיא טעונה לזיכרון בזמן ריצתה. כל תוכנית מקור (התוכנית כפי שנכתבה בידי המתכנת), תתורגם בסופו של דבר באמצעות תוכנה מיוחדת לצורה סופית זו.

היע"מ יודע לבצע קוד שנמצא בפורמט של **שפת מכונה**. זהו רצף של ביטים, המהווים קידוד בינארי של סדרת הוראות המכונה המרכיבות את התוכנית. קוד כזה אינו קריא למשתמש, ולכן לא נוה לקודד (או לקרוא) תוכניות ישירות בשפת מכונה. **שפת אסמבלי** (assembly language) היא שפת תכנות מאפשרת לייצג את הוראות המכונה בצורה סימבולית קלה ונוחה יותר לשימוש. כמובן שיש צורך לתרגם את הייצוג הסימבולי לקוד בשפת מכונה, כדי שהתוכנית תוכל לרוץ במחשב. תרגום זה נעשה באמצעות כלי שנקרא **אסמבלר** (assembler).

כידוע, לכל שפת תכנות עילית יש מהדר (compiler), או מפרש (interpreter), המתרגם תוכניות מקור לשפת מכונה. האסמבלר משמש בתפקיד דומה עבור שפת אסמבלי.

לכל מודל של יע"מ (כלומר לכל אירגון של מחשב) יש שפת מכונה יעודית משלו, ובהתאם גם שפת אסמבלי יעודית משלו. לפיכך, גם האסמבלר (כלי התרגום) הוא יעודי ושונה לכל יע"מ.

תפקידו של האסמבלר הוא לבנות קובץ המכיל קוד מכונה, מקובץ נתון של תוכנית הכתובה בשפת אסמבלי. זהו השלב הראשון במסלול אותו עוברת התוכנית, עד לקבלת קוד המוכן לריצה על חומרת המחשב. השלבים הבאים הם קישור (linkage) וטעינה (loading), אך בהם לא נעסוק בממ"ץ זה.

המשימה בפרויקט זה היא לכתוב אסמבלר (כלומר תוכנית המתרגמת לשפת מכונה), עבור שפת אסמבלי שנגדיר כאן במיוחד לצורך הפרויקט.

לתשומת לב: בהסברים הכלליים על אופן עבודת תוכנת האסמבלר, תהיה מדי פעם התייחסות גם לעבודת שלבי הקישור והטעינה. התייחסויות אלה נועדו על מנת לאפשר לכם להבין את המושך תהליך העיבוד של הפלט של תוכנת האסמבלר. אין לטעות: עליכם לכתוב את תוכנית האסמבלר בלבד. אין לכתוב את תוכניות הקישור והטעינה!!!

המחשב הדמיוני ושפת האסמבלי

נגדיר עתה את שפת האסמבלי ואת מודל המחשב הדמיוני, עבור פרויקט זה.

הערה: תאור מודל המחשב להלן הוא חלקי בלבד, ככל שנחוץ לביצוע המשימות בפרויקט.

"חומרה":

המחשב בפרויקט מורכב מ**מעבד** (יע"מ), **אוגרים** (רגיסטרים), **זיכרון** RAM. חלק מהזיכרון משמש כמחסנית (stack).

למעבד 16 אוגרים כלליים, בשמות: $r0, r1, r2, r3, r4, r5, r6, r7, \dots, r15$. גודלו של כל אוגר הוא 20 סיביות. הסיבית הכי פחות משמעותית תצוין כסיבית מס' 0, והסיבית המשמעותית ביותר כמס' 19. שמות האוגרים נכתבים תמיד עם אות 'r' קטנה.

כמו כן יש במעבד אוגר בשם PSW (program status word), המכיל מספר דגלים המאפיינים את מצב הפעילות במעבד בכל רגע נתון. ראו בהמשך, בתיאור הוראות המכונה, הסברים לגבי השימוש בדגלים אלו.

גודל הזיכרון הוא 8192 תאים, בכתובות 0-8191, וכל תא הוא בגודל של 20 סיביות. לתא בזיכרון נקרא גם בשם **"מילה"**. הסיביות בכל מילה ממוספרות כמו באוגר.

מחשב זה עובד רק עם מספרים שלמים חיוביים ושליילים. אין תמיכה במספרים ממשיים. האריתמטיקה נעשית בשיטת המשלים ל-2 (2's complement). כמו כן יש תמיכה בתווים (characters), המיוצגים בקוד ascii.

מבנה הוראת המכונה:

כל הוראת מכונה במודל שלנו מורכבת מפעולה ואופרנדים. מספר האופרנדים הוא בין 0 ל-2, בהתאם לסוג הפעולה. מבחינת התפקיד של כל אופרנד, נבחין בין אופרנד מקור (source) ואופרנד יעד (destination).

כל הוראת מכונה מקודדת למספר מילות זיכרון רצופות, **החל ממילה אחת ועד למקסימום שש מילים**, בהתאם לסוג הפעולה (ראו פרטים בהמשך).

בקובץ הפלט המכיל את קוד המכונה שבונה האסמבלר, כל מילה תקודד "בבסיס מיוחד" (ראו פרטים לגבי קבצי פלט בהמשך).

בהוראת מכונה ללא אופרנדים, המבנה של המילה הראשונה (והיחידה) הוא:

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	A	R	E	opcode															

ואילו בהוראות עם אופרנדים המבנה של הקידוד יכיל לפחות 2 מילות זיכרון במבנה הבא:

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	A	R	E	opcode															
0	A	R	E	funct				אוגר מקור				מיעון מקור		אוגר יעד				מיעון יעד	
מילים נוספות בהתאם לשיטות המיעון																			

במודל המכונה שלנו יש 16 פעולות, בפועל, למרות שניתן לקודד יותר פעולות. כל פעולה מיוצגת בשפת אסמבלי באופן סימבולי על ידי **שם-פעולה**, ובקוד המכונה על ידי קומבינציה ייחודית של ערכי שני שדות במילה הראשונה של ההוראה: **קוד-הפעולה (opcode)**, ו**פונקציה (funct)**.

להלן טבלת הפעולות:

שם הפעולה	funct (בבסיס עשרוני)	opcode (בבסיס עשרוני)
mov		0
cmp		1
add	10	2
sub	11	2
lea		4
clr	10	5
not	11	5
inc	12	5
dec	13	5
jmp	10	9
bne	11	9
jsr	12	9
red		12
prn		13
rts		14
stop		15

הערה: שם-הפעולה נכתב תמיד באותיות קטנות. פרטים על מהות הפעולות השונות יובאו בהמשך.

להלן מפרט השדות בקידוד הראשונה בקוד המכונה של כל הוראה.

שדה opcode: שדה זה מיוצג ב-16 סיביות, והוא מכיל ביט דולק בודד בהתאם לקוד הפעולה. למשל אם מדובר על קוד פעולה 9, אזי סיבית 9 תקבל ערך של 1.

סיבית 19: לא בשימוש, ערכה קבוע לאפס.

סיביות 16-18: עבור קידוד הוראות, סיביות אלה מכילות את סיווג הקידוד (A=Absolute, R=Relocatable, E=External) לכל מילת קידוד של הוראה יש סיווג, ובהתאם לסיווג הסיבית המתאימה בשדה ARE תקבל ערך 1.

סיביות 12-15: שדה זה, הנקרא **funct**, מתפקד כאשר מדובר בפעולה שקוד-הפעולה (opcode) שלה משותף לכמה פעולות שונות (כאמור, קודי-פעולה 2, 5 או 9). השדה funct יכול ערך ייחודי לכל פעולה מקבוצת הפעולות שיש להן אותו קוד-פעולה. אם קוד-הפעולה משמש לפעולה אחת בלבד, הסיביות של השדה funct יהיו מאופסות.

סיביות 8-11: מכילות את מספרו של אוגר המקור במידה ואופרנד המקור הוא אוגר. אם אין בהוראה אופרנד מקור שהוא אוגר, סיביות אלה יהיו מאופסות.

סיביות 6-7: מכילות את מספרה של שיטת המיעון של אופרנד המקור. אם אין בהוראה אופרנד מקור, סיביות אלה יהיו מאופסות. מפרט של שיטות המיעון השונות יינתן בהמשך.

סיביות 2-5: מכילות את מספרו של אוגר היעד במידה ואופרנד היעד הוא אוגר. אם אין בהוראה אופרנד יעד שהוא אוגר, סיביות אלה יהיו מאופסות.

סיביות 0-1: מכילות את מספרה של שיטת המיעון של אופרנד היעד. אם אין בהוראה אופרנד יעד, סיביות אלה יהיו מאופסות.

שיטות מיעון:

שיטות מיעון (addressing modes) הן האופנים השונים בהם ניתן להעביר אופרנדים של הוראה מכונה. בשפת האסמבלי שלנו קיימות ארבע שיטות מיעון, המסומנות במספרים 0,1,2,3.

השימוש בשיטות המיעון מצריך מילות-מידע נוספות בקוד המכונה של הוראה, בנוסף למילים הראשונות. קידוד אופרנד של הוראה עשוי לייצר מילות זיכרון נוספות בהתאם לסוג שיטת המיעון. כאשר בהוראה יש שני אופרנדים, קודם יופיעו מילות-המידע הנוספות של אופרנד המקור, ולאחריהם מילות-המידע הנוספות של אופרנד היעד.

להלן המפרט של שיטות המיעון.

מספר	שיטת המיעון	תוכן מילות-המידע הנוספות	תחביר האופרנד באסמבלי	דוגמה
0	מיעון מיידי (immediate)	מילת-מידע נוספת של ההוראה מכילה את האופרנד עצמו, שהוא מספר שלם בשיטת המשלים ל-2, ברוחב של 16 סיביות מילה זו תסווג מסוג A	האופרנד מתחיל בתו # ולאחריו ובצמוד אליו מופיע מספר שלם בבסיס עשרוני.	mov #-1, r2 בדוגמה זו האופרנד הראשון של ההוראה (אופרנד המקור) נתון בשיטת מיעון מיידי. ההוראה כותבת את הערך -1 אל אוגר r2.
1	מיעון ישיר (direct)	2 מילות-מידע נוספות של ההוראה יכילו את כתובת האופרנד במבנה של כתובת בסיס והיסט. <u>כתובת בסיס = כתובת הקרובה ביותר לכתובת האופרנד, הקטנה ממנו, ומתחלקת ב-16. למשל, אם כתובת האופרנד היא 36, אזי כתובת הבסיס היא 32, מאחר ו-32 הוא המספר הקרוב ביותר ל-36 שקטן ממנו ומתחלק ב-16.</u> <u>היסט = המרחק מכתובת הבסיס לכתובת האופרנד.</u> בדוגמה שניתנה בהסבר לכתובת בסיס, ההיסט יהיה 4. מאחר והמרחק (ההיסט) מ-32 ל-36 הוא 4. מילת-המידע הנוספת הראשונה תכיל את כתובת הבסיס. ומילת המידע השנייה תכיל את ההיסט. כתובת הבסיס וההיסט מיוצגים כמספר ללא סימן ברוחב של 16 סיביות. והם יסווגו מסוג R במידה והאופרנד הוא תווית חיצונית, כתובת הבסיס וההיסט יכילו אפסים, וקידודים אלה יסווגו מסוג E במקרה כזה.	האופרנד הוא <u>תווית</u> שכבר הוגדרה, או שתוגדר בהמשך הקובץ. ההגדרה נעשית על ידי כתיבת התווית בתחילת השורה של הנחית ".data" או ".string", או בתחילת השורה של הוראה, או באמצעות אופרנד של הנחית ".extern". התווית מייצגת באופן סימבולי כתובת בזיכרון.	השורה הבאה מגדירה את התווית x: x: .data 23 ההוראה: dec x מקטינה ב-1 את תוכן המילה שבכתובת x בזיכרון ("משתנה" x). הכתובת של x מקודדת במילות-המידע הנוספות, במבנה של כתובת בסיס והיסט. <u>דוגמה נוספת:</u> ההוראה jmp next מבצעת קפיצה אל השורה בה מוגדרת התווית next (כלומר ההוראה הבאה שתבצע נמצאת בכתובת next). הכתובת next מקודדת במילות-המידע הנוספות, במבנה של כתובת בסיס והיסט.

מספר	שיטת המיעון	תוכן מילות-המידע הנוספות	תחביר האופרנד באסמבלי	דוגמה
2	מיעון אינדקס	בשיטה זו, יש בקידוד ההוראה 2 מילות מידע נוספות, המכילות את כתובת התווית בצורת כתובת בסיס והיסט כפי שתואר בשיטת מיעון מספר 1. מספרו של האוגר שצוין בסוגריים המרובעות, יישמר כפי שמוסבר בשיטת מיעון מספר 3, בסיביות אוגר המקור/יעד בהתאם לכך אם האופרנד הוא אופרנד מקור/יעד.	האופרנד מתחיל בשם של תווית ולאחריה בסוגריים מרובעות, שם של אוגר שמספרו בין 10 ל- 15 בלבד.	השורה הבאה מגדירה את התווית x: <pre>x: .data 23,12,34,50</pre> הפקודה: <pre>mov x[r12], r4</pre> בדוגמה זו האופרנד הראשון הוא גישה לכתובת של X בזיכרון, והחל משם מתקדמים כמות של מילות זיכרון נוספות לפי הערך שיהיה בזמן ריצה באוגר r12, ומה שיש באותה כתובת יישמש את המעבד כאופרנד המקור. בדוגמה זו אופרנד זו יישמר באוגר r4
3	מיעון אוגר ישיר (register direct)	אין מילות-מידע נוספות בגין האוגר. מספרו של האוגר יישמר בסיביות של אוגר המקור/יעד בהתאם לכך אם האופרנד הוא אופרנד מקור/יעד.	האופרנד הוא שם של אוגר.	<pre>clr r1</pre> בדוגמה זו, ההוראה clr מאפסת את האוגר r1. <u>דוגמה נוספת:</u> <pre>mov #-1, r2</pre> האופרנד השני של ההוראה (אופרנד היעד) נתון בשיטת מיעון אוגר ישיר. ההוראה כותבת את הערך המיידני -1 אל אוגר r2.

מפרט הוראות המכונה:

בתיאור הוראות המכונה נשתמש במונח **PC** (קיצור של "Program Counter"). זהו אוגר פנימי של המעבד (לא אוגר כללי), שמכיל בכל רגע נתון את כתובת הזיכרון בה נמצאת **ההוראה הנוכחית שמתבצעת** (הכוונה תמיד לכתובת המילה הראשונה של ההוראה).

הוראות המכונה מתחלקות לשלוש קבוצות, לפי מספר האופרנדים הנדרשים לפעולה.

קבוצת ההוראות הראשונה:

אלו הן הוראות המקבלות שני אופרנדים.

ההוראות השייכות לקבוצה זו הן: mov, cmp, add, sub, lea

הוראה	opcode	funct	הפעולה המתבצעת	דוגמה	הסבר הדוגמה
mov	0		מבצעת העתקה של תוכן אופרנד המקור (האופרנד הראשון) אל אופרנד היעד (האופרנד השני).	mov A, r1	העתק את תוכן המשתנה A (המילה שבכתובת A בזיכרון) אל אוגר r1.
cmp	1		מבצעת השוואה בין שני האופרנדים. ערך אופרנד היעד (השני) מופחת מערך אופרנד המקור (הראשון), ללא שמירת תוצאת החיסור. פעולת החיסור מעדכנת דגל בשם Z ("דגל האפס") באוגר הסטטוס (PSW).	cmp A, r1	אם תוכן המשתנה A זהה לתוכנו של אוגר r1 אזי הדגל Z ("דגל האפס") באוגר הסטטוס (PSW) יודלק, אחרת הדגל יאופס.
add	2	10	אופרנד היעד (השני) מקבל את תוצאת החיבור של אופרנד המקור (הראשון) והיעד (השני).	add A, r0	אוגר r0 מקבל את תוצאת החיבור של תוכן המשתנה A ותוכנו הנוכחי של r0.
sub	2	11	אופרנד היעד (השני) מקבל את תוצאת החיסור של אופרנד המקור (הראשון) מאופרנד היעד (השני).	sub #3, r1	אוגר r1 מקבל את תוצאת החיסור של הקבוע 3 מתוכנו הנוכחי של האוגר r1.
lea	4		lea הוא קיצור (ראשי תיבות) של load effective address. פעולה זו מציבה את מען הזיכרון המיוצג על ידי התווית שבאופרנד הראשון (המקור), אל האופרנד השני (היעד).	lea HELLO, r1	המען שמייצגת התווית HELLO מוצב לאוגר r1.

קבוצת ההוראות השנייה:

אלו הן הוראות המקבלות אופרנד אחד בלבד. אופן הקידוד של האופרנד הוא כמו של אופרנד היעד בהוראה עם שני אופרנדים. השדה של אופרנד המקור (סיביות 2-3) במילה הראשונה בקידוד ההוראה אינו בשימוש, ולפיכך יהיו מאופס.

ההוראות השייכות לקבוצה זו הן: clr, not, inc, dec, jmp, bne, jsr, red, prn

הוראה	opcode	funct	הפעולה המתבצעת	דוגמה	הסבר הדוגמה
clr	5	10	איפוס תוכן האופרנד.	clr r2	האוגר r2 מקבל את הערך 0.
not	5	11	היפוך הסיביות באופרנד (כל סיבית שערכה 0 תהפוך ל-1 ולהיפך: 1 ל-0).	not r2	כל ביט באוגר r2 מתהפך.
inc	5	12	הגדלת תוכן האופרנד באחד.	inc r2	תוכן האוגר r2 מוגדל ב-1.
dec	5	13	הקטנת תוכן האופרנד באחד.	dec Count	תוכן המשתנה Count מוקטן ב-1.
jmp	9	10	קפיצה (הסתעפות) בלתי מותנית אל ההוראה שנמצאת במען המיוצג על ידי האופרנד. כלומר, כתוצאה מביצוע ההוראה, מצביע התוכנית (PC) מקבל את כתובת יעד הקפיצה.	jmp Line	PC ← PC + addressOf(Line) הכתובת של תווית Line נשמרת לתוך מצביע התוכנית ולפיכך ההוראה הבאה שתבצע תהיה במען Line.

הוראה	opcode	funct	הפעולה המתבצעת	דוגמה	הסבר הדוגמה
bne	9	11	bne הוא קיצור (ראשי תיבות) של: branch if not equal (to zero). זוהי הוראת הסתעפות מותנית. אם ערכו של הדגל Z באוגר הסטטוס (PSW) הינו 0, אזי מצביע התוכנית (PC) מקבל את כתובת יעד הקפיצה. כזכור, הדגל Z נקבע באמצעות הוראת cmp.	bne Line	אם ערך הדגל Z באוגר הסטטוס (PSW) הוא 0, אזי $PC \leftarrow \text{address}(\text{Line})$ מצביע התוכנית יקבל את כתובת התווית Line, ולפיכך ההוראה הבאה שתבצע תהיה במען Line.
jsr	9	12	קריאה לשגרה (סברוטניה). כתובת ההוראה שאחרי הוראת jsr הנוכחית ($PC+2$) נדחפת לתוך המחסנית שבזיכרון המחשב, ומצביע התוכנית (PC) מקבל את כתובת השגרה. הערה: חזרה מהשגרה מתבצעת באמצעות הוראת rts, תוך שימוש בכתובת שבמחסנית.	jsr SUBR	$\text{push}(PC+2)$ $PC \leftarrow \text{address}(\text{SUBR})$ מצביע התוכנית יקבל את כתובת התווית SUBR, ולפיכך, ההוראה הבאה שתבצע תהיה במען SUBR. כתובת החזרה מהשגרה נשמרת במחסנית.
red	12		קריאה של תו מהקלט הסטנדרטי (stdin) אל האופרנד.	red r1	קוד ה-ascii של התו הנקרא מהקלט ייכנס לאוגר r1.
prn	13		הדפסת התו הנמצא באופרנד, אל הפלט הסטנדרטי (stdout).	prn r1	יודפס לפלט התו (קוד ascii) הנמצא באוגר r1.

קבוצת ההוראות השלישית:

אלו הן הוראות ללא אופרנדים. קידוד ההוראה מורכב ממילה אחת בלבד. השדות של אופרנד המקור ושל אופרנד היעד (סיביות 0-3) במילה הראשונה של ההוראה אינם בשימוש, ולפיכך יהיו מאופסים.

ההוראות השייכות לקבוצה זו הן: rts, stop.

הוראה	opcode	הפעולה המתבצעת	דוגמה	הסבר הדוגמה
rts	14	מתבצעת חזרה משיגרה. הערך שבראש המחסנית של המחשב מוצא מן המחסנית, ומוכנס למצביע התוכנית (PC). הערה: ערך זה נכנס למחסנית בקריאה לשגרה ע"י הוראת jsr.	rts	$PC \leftarrow \text{pop}()$ ההוראה הבאה שתבצע תהיה זו שאחרי הוראת jsr שקראה לשגרה.
stop	15	עצירת ריצת התוכנית.	stop	התוכנית עוצרת מיידית.

מבנה תכנית בשפת אסמבלי:

תכנית בשפת אסמבלי בנויה ממקראים וממשפטים (statements).

מקראים:

מקראים הם קטעי קוד הכוללים בתוכם משפטים. בתוכנית ניתן להגדיר מקרו ולהשתמש בו במקומות שונים בתוכנית. השימוש במקרו ממקום מסוים בתוכנית יגרום לפרישת המקרו לאותו מקום.

הגדרת מקרו נעשית באופן הבא: (בדוגמה שם המקרו הוא m1)

```
macro m1
  inc r2
  mov A,r1
endm
```

שימוש במקרו הוא פשוט אזכור שמו.
למשל, אם בתוכנית במקום כלשהו כתוב:

```
*  
*  
*  
m1  
*  
*  
m1  
*  
*  
*
```

בדוגמה זו, השתמשנו פעמיים במקרו m1, התוכנית לאחר פרישת המקרו תיראה כך:

```
*  
*  
*  
*  
inc r2  
mov A,r1  
*  
*  
inc r2  
mov A,r1  
*  
*  
*
```

התוכנית לאחר פרישת המקרו היא התוכנית שהאסמבלר אמור לתרגם.

הנחיות לגבי מקרו:

- אין במערכת הגדרות מקרו מקוננות.
- שם של הוראה או הנחיה לא יכול להיות שם של מקרו.
- ניתן להניח שלכל שורת מקרו בקוד המקור קיימת סגירה עם שורת endm (אין צורך לבדוק זאת).
- הגדרת מקרו תהיה תמיד לפני הקריאה למקרו
- נדרש שהקדם-אסמבלר ייצור קובץ עם הקוד המורחב הכולל פרישה של המקרו (הרחבה של קובץ המקור המתואר בהמשך). "קובץ המקור המורחב" הוא "קובץ מקור" לאחר פרישת המקרו, לעומת "קובץ מקור ראשוני" שהוא קובץ הקלט למערכת, כולל הגדרת המקרואים.

משפטים:

קובץ מקור בשפת אסמבלי מורכב משורות המכילות משפטים של השפה, כאשר כל משפט מופיע בשורה נפרדת. כלומר, ההפרדה בין משפט למשפט בקובץ המקור הינה באמצעות התו "\n" (שורה חדשה).

אורכה של שורה בקובץ המקור הוא 80 תווים לכל היותר (לא כולל התו \n).

יש ארבעה סוגי משפטים (שורות בקובץ המקור) בשפת אסמבלי, והם:

סוג המשפט	הסבר כללי
משפט ריק	זוהי שורה המכילה אך ורק תווים לבנים (whitespace). כלומר רק את התווים ' ' ו- '\t' (רווחים וטאבים). ייתכן ובשורה אין אף תו (למעט התו \n), כלומר השורה ריקה.
משפט הערה	זוהי שורה בה התו הראשון הינו ';' (נקודה פסיק). על האסמבלר להתעלם לחלוטין משורה זו.
משפט הנחיה	זהו משפט המנחה את האסמבלר מה עליו לעשות כשהוא פועל על תוכנית המקור. יש מספר סוגים של משפטי הנחיה. משפט הנחיה עשוי לגרום להקצאת זיכרון ואתחול משתנים של התוכנית, אך הוא אינו מייצר קידוד של הוראות מכונה המיועדות לביצוע בעת ריצת התוכנית.
משפט הוראה	זהו משפט המייצר קידוד של הוראות מכונה לביצוע בעת ריצת התוכנית. המשפט מורכב משם ההוראה (פעולה) שעל המעבד לבצע, והאופרנדים של ההוראה.

כעת נפרט יותר לגבי סוגי המשפטים השונים.

משפט הנחיה:

משפט הנחיה הוא בעל המבנה הבא:

בתחילת המשפט יכולה להופיע הגדרה של תווית (label). לתווית יש תחביר חוקי שיתואר בהמשך. התווית היא אופציונאלית.

לאחר מכן מופיע שם ההנחיה. לאחר שם ההנחיה יופיעו פרמטרים (מספר הפרמטרים בהתאם להנחיה).

שם של הנחיה מתחיל בתו '.' (נקודה) ולאחריו תווים באותיות קטנות (lower case) בלבד.

יש ארבעה סוגים (שמות) של משפטי הנחיה, והם:

1. ההנחיה 'data'.

הפרמטרים של ההנחיה 'data' הם מספרים שלמים חוקיים (אחד או יותר) המופרדים על ידי התו ',', (פסיק). לדוגמה:

data 7, -57, +17, 9

יש לשים לב שהפסיקים אינם חייבים להיות צמודים למספרים. בין מספר לפסיק ובין פסיק למספר יכולים להופיע רווחים וטאבים בכל כמות (או בכלל לא), אולם הפסיק חייב להופיע בין המספרים. כמו כן, אסור שיופיע יותר מפסיק אחד בין שני מספרים, וגם לא פסיק אחרי המספר האחרון או לפני המספר הראשון.

המשפט 'data' מנחה את האסמבלר להקצות מקום בתמונת הנתונים (data image), אשר בו יאוחסנו הערכים של הפרמטרים, ולקדם את מונה הנתונים, בהתאם למספר הערכים. אם בהנחית data מוגדרת תווית, אזי תווית זו מקבלת את ערך מונה הנתונים (לפני הקידום), ומוכנסת אל טבלת הסמלים. דבר זה מאפשר להתייחס אל מקום מסוים בתמונת הנתונים דרך שם התווית (למעשה, זוהי דרך להגדיר שם של משתנה).

כלומר אם נכתוב:

XYZ: data 7, -57, +17, 9

אזי יוקצו בתמונת הנתונים ארבע מילים רצופות שיכילו את המספרים שמופיעים בהנחיה. התווית XYZ מזוהה עם כתובת המילה הראשונה.

אם נכתוב בתוכנית את ההוראה:

mov XYZ, r1

אזי בזמן ריצת התוכנית יוכנס לאוגר r1 הערך 7.

ואילו ההוראה :

```
lea XYZ, r1
```

תכניס לאוגר r1 את ערך התווית XYZ (כלומר הכתובת בזיכרון בה מאוחסן הערך 7).

2. ההנחיה '.string'

להנחיה '.string' פרמטר אחד, שהוא מחרוזת חוקית. תווי המחרוזת מקודדים לפי ערכי ה-ascii המתאימים, ומוכנסים אל תמונת הנתונים לפי סדרם, כל תו במילה נפרדת. בסוף המחרוזת יתווסף התו '0' (הערך המספרי 0), המסמן את סוף המחרוזת. מונה הנתונים של האסמבלר יקודם בהתאם לאורך המחרוזת (בתוספת מקום אחד עבור התו המסיים). אם בשורת ההנחיה מוגדרת תווית, אזי תווית זו מקבלת את ערך מונה הנתונים (לפני הקידום) ומוכנסת אל טבלת הסמלים, בדומה כפי שנעשה עבור '.data' (כלומר ערך התווית יהיה הכתובת בזיכרון שבה מתחילה המחרוזת).

לדוגמה, ההנחיה :

```
STR: .string "abcdef"
```

מקצה בתמונת הנתונים רצף של 7 מילים, ומאתחלת את המילים לקודי ה-ascii של התווים לפי הסדר במחרוזת, ולאחריהם הערך 0 לסימון סוף מחרוזת. התווית STR מזוהה עם כתובת התחלת המחרוזת.

3. ההנחיה '.entry'

להנחיה '.entry' פרמטר אחד, והוא שם של תווית המוגדרת בקובץ המקור הנוכחי (כלומר תווית שמקבלת את ערכה בקובץ זה). מטרת ההנחיה entry היא לאפיין את התווית הזו באופן שיאפשר לקוד אסמבלי הנמצא בקבצי מקור אחרים להשתמש בה (כאופרנד של הוראה).

לדוגמה, השורות :

```
HELLO: .entry HELLO
        add #1, r1
```

מודיעות לאסמבלר שאפשר להתייחס בקובץ אחר לתווית HELLO המוגדרת בקובץ הנוכחי.

לתשומת לב : תווית המוגדרת בתחילת שורת entry. הינה חסרת משמעות והאסמבלר **מתעלם** מתווית זו (אפשר שהאסמבלר יוציא הודעת אזהרה).

4. ההנחיה '.extern'

להנחיה '.extern' פרמטר אחד, והוא שם של תווית שאינה מוגדרת בקובץ המקור הנוכחי. מטרת ההוראה היא להודיע לאסמבלר כי התווית מוגדרת בקובץ מקור אחר, וכי קוד האסמבלי בקובץ הנוכחי עושה בתווית שימוש.

נשים לב כי הנחיה זו תואמת להנחית '.entry' המופיעה בקובץ בו מוגדרת התווית. בשלב הקישור תתבצע התאמה בין ערך התווית, כפי שנקבע בקוד המכונה של הקובץ שהגדיר את התווית, לבין קידוד ההוראות המשתמשות בתווית בקבצים אחרים (שלב הקישור אינו רלוונטי לממ"ן זה).

לדוגמה, משפט ההנחיה '.extern' התואם למשפט ההנחיה '.entry' מהדוגמה הקודמת יהיה :

```
.extern HELLO
```

הערה : לא ניתן להגדיר באותו הקובץ את אותה התווית גם כ-entry וגם כ-extern (בדוגמאות לעיל, התווית HELLO).

לתשומת לב: תווית המוגדרת בתחילת שורת extern. הינה חסרת משמעות והאסמבלר מתעלם מתווית זו (אפשר שהאסמבלר יוציא הודעת אזהרה).

משפט הוראה:

משפט הוראה מורכב מהחלקים הבאים:

1. תווית אופציונלית.
2. שם הפעולה.
3. אופרנדים, בהתאם לסוג הפעולה (בין 0 ל-2 אופרנדים).

אם מוגדרת תווית בשורת ההוראה, אזי היא תוכנס אל טבלת הסמלים. ערך התווית יהיה מען המילה הראשונה של ההוראה בתוך תמונת הקוד שבונה האסמבלר.

שם הפעולה תמיד באותיות קטנות (lower case), והוא אחת מ-16 הפעולות שפורטו לעיל.

לאחר שם הפעולה יופיעו האופרנדים, בהתאם לסוג הפעולה. יש להפריד בין שם הפעולה לבין האופרנד הראשון באמצעות רווחים ו/או טאבים (אחד או יותר).

כאשר יש שני אופרנדים, האופרנדים מופרדים זה מזה בתו ' ', (פסיק). בדומה להנחיה 'data', **לא חייבת להיות הצמדה של האופרנדים לפסיק**. כל כמות של רווחים ו/או טאבים משני צידי הפסיק היא חוקית.

למשפט הוראה עם שני אופרנדים המבנה הבא:

label: opcode source-operand, target-operand

לדוגמה:

HELLO: add r7, B

למשפט הוראה עם אופרנד אחד המבנה הבא:

label: opcode target-operand

לדוגמה:

HELLO: bne XYZ

למשפט הוראה ללא אופרנדים המבנה הבא:

label: opcode

לדוגמה:

END: stop

אפיון השדות במשפטים של שפת האסמבלי

תווית:

תווית היא סמל שמוגדר בתחילת משפט הוראה או בתחילת הנחית data או string. תווית חוקית מתחילה באות אלפביתית (גדולה או קטנה), ולאחריה סדרה כלשהי של אותיות אלפביתיות (גדולות או קטנות) ו/או ספרות. האורך המקסימלי של תווית הוא 31 תווים.

הגדרה של תווית מסתיימת בתו ' ': (נקודתיים). תו זה אינו מהווה חלק מהתווית, אלא רק סימן המציין את סוף ההגדרה. התו ' ': חייב להיות צמוד לתווית (ללא רווחים).

אסור שאותה תווית תוגדר יותר מפעם אחת (כמובן בשורות שונות). אותיות קטנות וגדולות נחשבות שונות זו מזו.

לדוגמה, התוויות המוגדרות להלן הן תוויות חוקיות.

hEllo:
x:
He78902:

לתשומת לב: מילים שמורות של שפת האסמבלי (כלומר שם של פעולה או הנחיה, או שם של אוגר) אינן יכולות לשמש גם כשם של תווית. לדוגמה: הסמלים `r3`, `add` לא יכולים לשמש כתוויות, אבל הסמלים `R3`, `r19`, `Add` הם תוויות חוקיות.

התווית מקבלת את ערכה בהתאם להקשר בו היא מוגדרת. תווית המוגדרת בהנחיות `.data` או `.string`, תקבל את ערך מונה הנתונים (`data counter`) הנוכחי, בעוד שתווית המוגדרת בשורת הוראה תקבל את ערך מונה ההוראות (`instruction counter`) הנוכחי.

לתשומת לב: מותר במשפט הוראה להשתמש באופרנד שהוא סמל שאינו מוגדר כתווית בקובץ הנוכחי, כל עוד הסמל מאופיין כחיצוני (באמצעות הנחית `extern`. כלשהי בקובץ הנוכחי).

מספר:

מספר חוקי מתחיל בסימן אופציונלי: '-' או '+' ולאחריו סדרה של ספרות בבסיס עשרוני. לדוגמה: `+123`, `-5`, `76`. הם מספרים חוקיים. אין תמיכה בשפת האסמבלי שלנו בייצוג בבסיס אחר מאשר עשרוני, ואין תמיכה במספרים שאינם שלמים.

מחרוזת:

מחרוזת חוקית היא סדרת תווי `ascii` נראים (שניתנים להדפסה), המוקפים במרכאות כפולות (המרכאות אינן נחשבות חלק מהמחרוזת). דוגמה למחרוזת חוקית: `"hello world"`.

סימון המילים בקוד המכונה באמצעות המאפיין "A,R,E"

האסמבלר בונה מלכתחילה קוד מכונה שמיועד לטעינה החל מכתובת 100. אולם, לא בכל פעם שהקוד ייטען לזיכרון לצורך הרצה, מובטח שאפשר יהיה לטעון אותו החל מכתובת 100. במקרה כזה, קוד המכונה הנתון אינו מתאים ויש צורך לתקן אותו. לדוגמה, מילת-המידע של אופרנד בשיטת מיעון ישיר לא תהיה נכונה, כי הכתובת השתנתה.

הרעיון הוא להכניס תיקונים נקודתיים בקוד המכונה בכל פעם שייטען לזיכרון לצורך הרצה. כך אפשר יהיה לטעון את הקוד בכל פעם למקום אחר, בלי צורך לחזור על תהליך האסמבלי. תיקונים כאלה נעשים בשלב הקישור והטעינה של הקוד (אנו לא מטפלים בכך בממ"ן זה), אולם על האסמבלר להוסיף מידע בקוד המכונה שיאפשר לזהות את הנקודות בקוד בהן נדרש תיקון.

לצד כל מילה בקוד המכונה, האסמבלר מוסיף מאפיין שנקרא `"A,R,E"`. לכל מילה בקוד, מוצמד שדה המכיל את אחת האותיות `A` או `R` או `E`.

- האות `A` (קיצור של `Absolute`) באה לציין שתוכן המילה אינו תלוי במקום בזיכרון בו ייטען בפועל קוד המכונה של התוכנית בעת ביצועה (למשל, 2 המילים המכילים את קוד ההוראה ואת שיטות המיעון, או מילת-מידע המכילה אופרנד מיידי).
- האות `R` (קיצור של `Relocatable`) באה לציין שתוכן המילה תלוי במקום בזיכרון בו ייטען בפועל קוד המכונה של התוכנית בעת ביצועה (למשל, מילות-מידע המכילות כתובת של תווית בצורת כתובת בסיס והיסט).
- האות `E` (קיצור של `External`) באה לציין שתוכן המילה תלוי בערכו של סמל שאינו מוגדר בקובץ המקור הנוכחי (למשל, מילת-מידע המכילה ערך של סמל המופיע בהנחיית `extern`).

נשים לב כי רוב המילים בקוד המכונה מאופיינות על ידי האות `A`. למעשה, רק מילות-המידע הנוספות של שיטת מיעון ישיר ושל שיטת מיעון אינדקס מאופיינות על ידי האות `R` או `E` (תלוי אם האופרנד בקוד האסמבלי הוא תווית מקומית או סמל חיצוני).

כאשר האסמבלר מקבל כקלט תוכנית בשפת אסמבלי, עליו לטפל תחילה בפרישת המקרואים, ורק לאחר מכן לעבור על התוכנית אליה נפרשו המקרואים. כלומר, פרישת המקרואים תעשה בשלב "קדם אסמבלר", לפני שלב האסמבלר (המתואר בהמשך). אם התוכנית אינה מכילה מקרו, תוכנית הפרישה תהיה זהה לתוכנית המקור.

דוגמה לשלב קדם אסמבלר. האסמבלר מקבל את התוכנית הבאה בשפת אסמבלי:

```
; file ps.as
.entry LIST
.extern W

MAIN:      add    r3, LIST
LOOP:      prn    #48
           macro m1
             inc r6
             mov r3, W
           endm
           lea    STR, r6
           m1
           sub    r1, r4
           bne    END
           cmp    val1, #-6
           bne    END[r15]
           dec    K
.entry MAIN
           sub    LOOP[r10], r14
END:       stop
STR:       .string "abcd"
LIST:      .data  6, -9
           .data  -100
.entry K
K:         .data  31
.extern val1
```

תחילה האסמבלר עובר על התוכנית ופורש את כל המקרואים הקיימים בה. רק אם תהליך זה מסתיים בהצלחה, ניתן לעבור לשלב הבא. בדוגמה זו, התוכנית לאחר פרישת המקרו תיראה כך:


```

; file ps.am
.entry LIST
.extern W

MAIN:      add    r3, LIST
LOOP:      prn    #48
           lea    STR, r6
           inc    r6
           mov    r3, W
           sub    r1, r4
           bne    END
           cmp    val1, #-6
           bne    END[r15]
           dec    K

.entry MAIN
           sub    LOOP[r10], r14
END:       stop
STR:       .string "abcd"
LIST:      .data  6, -9
           .data  -100

.entry K
K:         .data  31
.extern val1

```

קוד התכנית, לאחר הפרישה, ישמר בקובץ חדש, כפי שיוסבר בהמשך.

אלגוריתם שלדי של קדם האסמבלר

נציג להלן אלגוריתם שלדי לתהליך קדם האסמבלר. לתשומת לב: אין חובה להשתמש דווקא באלגוריתם זה:

1. קרא את השורה הבאה מקובץ המקור. אם נגמר הקובץ, עבור ל- 9 (סיום).
2. האם השדה הראשון הוא שם מקרו המופיע בטבלת המקרו (כגון m1)? אם כן, החלף את שם המקרו והעתק במקומו את כל השורות המתאימות מהטבלה לקובץ, חזור ל- 1. אחרת, המשך.
3. האם השדה הראשון הוא "macro" (התחלת הגדרת מקרו)? אם לא, עבור ל- 6.
4. הדלק דגל "יש macro".
5. (קיימת הגדרת מקרו) הכנס לטבלת שורות מקרו את שם המקרו (לדוגמה m1).
6. קרא את השורה הבאה מקובץ המקור. אם נגמר קובץ המקור, עבור ל- 9 (סיום). אם דגל "יש macro" דולק ולא זוהתה תווית **endm** הכנס את השורה לטבלת המקרו ומחק את השורה הנ"ל מהקובץ. אחרת (לא מקרו) חזור ל- 1.
7. האם זוהתה תווית **endm**? אם כן, מחק את התווית מהקובץ והמשך. אם לא, חזור ל- 6.
8. כבה דגל "יש macro". חזור ל- 1. (סיום שמירת הגדרת מקרו)
9. סיום: שמירת קובץ מקרו פרוש.

כעת לאחר פרישת כל המקרואים ניתן לעבור לשלב התרגום לקוד מכונה, שלב האסמבלר.

אסמבלר עם שני מעברים

במעבר הראשון של האסמבלר, יש לזהות את הסמלים (תוויות) המופיעים בתוכנית, ולתת לכל סמל ערך מספרי שהוא המען בזיכרון שהסמל מייצג. במעבר השני, באמצעות ערכי הסמלים, וכן קודי-הפעולה ומספרי האוגרים, בונים את קוד המכונה.

קוד המכונה של התוכנית (הוראות ונתונים) נבנה כך שיתאים לטעינה בזיכרון החל ממען 100 (עשרוני).
התרגום של תוכנית המקור שבדוגמה לקוד בינארי מוצג להלן:

Address (decimal)	Source Code	Machine Code (binary)																			
		19	-	-	-	-	-	-	-	-	-	9	8	7	6	5	4	3	2	1	0
0100	MAIN: add r3, LIST	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0101		0	1	0	0	1	0	1	0	0	0	1	1	1	1	0	0	0	0	0	1
0102		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
0103		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0104	LOOP: prn #48	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0105		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0106		0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
0107	lea STR, r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0108		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1	1
0109		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0110		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
0111	inc r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0112		0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	1	1
0113	mov r3, W	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0114		0	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	1
0115		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0116		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0117	sub r1, r4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0118		0	1	0	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	1	1
0119	bne END	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0120		0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1
0121		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0122		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0123	cmp val1, #-6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0124		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0125		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0126		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0127		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
0128	bne END[r15]	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0129		0	1	0	0	1	0	1	1	0	0	0	0	0	0	1	1	1	1	1	0
0130		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0131		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0132	dec K	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0133		0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	1
0134		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
0135		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
0136	sub LOOP[r10],r14	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0137		0	1	0	0	1	0	1	1	1	0	1	0	1	0	1	1	1	0	1	1
0138		0	0	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0
0139		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0140	END: stop	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0141	STR: .string "abcd"	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	1
0142		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	0
0143		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	1
0144		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0	0
0145		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0146	LIST: .data 6, -9	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0147		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
0148	.data -100	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	0	0
0149	K: .data 31	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

האסמבלר מחזיק טבלה שבה רשומים כל שמות הפעולה של ההוראות והקודים הבינאריים (opcode, funct) המתאימים להם, ולכן שמות הפעולות ניתנים להמרה לבינארי בקלות. כאשר נקרא שם פעולה, אפשר פשוט לעיין בטבלה ולמצוא את הקידוד הבינארי.

כדי לבצע המרה לבינארי של אופרנדים שכתובים בשיטות מיעון המשתמשות בסמלים (תוויות), יש צורך לבנות טבלה המכילה את ערכי כל הסמלים. אולם בהבדל מהקודים של הפעולות, הידועים מראש, הרי המענים בזיכרון עבור הסמלים שבשימוש התוכנית אינם ידועים, עד אשר תוכנית המקור נסרקה כולה ונתגלו כל הגדרות הסמלים.

למשל, בקוד לעיל, האסמבלר אינו יכול לדעת שהסמל END אמור להיות משויך למען 140 (עשרוני), ושהסמל K אמור להיות משויך למען 149, אלא רק לאחר שנקראו כל שורות התוכנית.

לכן מפרידים את הטיפול של האסמבלר בסמלים לשני שלבים. בשלב הראשון בונים טבלה של כל הסמלים, עם הערכים המספריים המשווים להם, ובשלב השני מחליפים את כל הסמלים, המופיעים באופרנדים של ההוראות התוכנית, בערכיהם המספריים. הביצוע של שני שלבים אלה כרוך בשתי סריקות (הנקראות "מעברים") של קובץ המקור.

במעבר הראשון נבנית טבלת סמלים בזיכרון, ובה לכל סמל שבתוכנית המקור משויך ערך מספרי, שהוא מען בזיכרון.

במעבר השני נעשית ההמרה של קוד המקור לקוד מכונה. בתחילת המעבר השני צריכים הערכים של כל הסמלים להיות כבר ידועים

עבור הדוגמה, טבלת הסמלים נתונה להלן. לכל סמל יש בטבלה גם מאפיינים (attributes) שיוסברו בהמשך. אין חשיבות לסדר השורות בטבלה (כאן הטבלה לפי הסדר בו הוגדרו הסמלים בתוכנית).

Symbol	Value (decimal)	Base address	offset	Attributes
W	0	0	0	external
MAIN	100	96	4	code, entry
LOOP	104	96	8	code
END	140	128	12	code
STR	141	128	13	data
LIST	146	144	2	data, entry
K	149	144	5	data, entry
val1	0	0	0	external

לתשומת לב: תפקיד האסמבלר, על שני המעברים שלו, לתרגם קובץ מקור לקוד בשפת מכונה. בגמר פעולת האסמבלר, התוכנית טרם מוכנה לטעינה לזיכרון לצורך ביצוע. קוד המכונה חייב לעבור לשלבי הקישור/טעינה, ורק לאחר מכן לשלב הביצוע (שלבים אלה אינם חלק מהממ"ץ).

המעבר הראשון

במעבר הראשון נדרשים כללים כדי לקבוע איזה מען ישויד לכל סמל. העיקרון הבסיסי הוא לספור את המקומות בזיכרון, אותם תופסות ההוראות. אם כל הוראה תיטען בזיכרון למקום העוקב להוראה הקודמת, תציין ספירה כזאת את מען ההוראה הבאה. הספירה נעשית על ידי האסמבלר ומחוזקת במונה ההוראות (IC). ערכו ההתחלתי של IC הוא 100 (עשרוני), ולכן קוד המכונה של ההוראה הראשונה נבנה כך שייטען לזיכרון החל ממען 100. ה-IC מתעדכן בכל שורת הוראה המקצה מקום בזיכרון. לאחר שהאסמבלר קובע מהו אורך ההוראה, ה-IC מוגדל במספר התאים (מילים) הנתפסים על ידי ההוראה, וכך הוא מצביע על התא הפנוי הבא.

כאמור, כדי לקודד את ההוראות בשפת מכונה, מחזיק האסמבלר טבלה, שיש בה קידוד מתאים לכל שם פעולה. בזמן התרגום מחליף האסמבלר כל שם פעולה בקידוד שלה. כמו כן, כל אופרנד מוחלף בקידוד מתאים, אך פעולת החלפה זו אינה כה פשוטה. ההוראות משתמשות בשיטות מיעון מגוונות לאופרנדים. אותה פעולה יכולה לקבל משמעויות שונות, בכל אחת משיטות המיעון, ולכן יתאימו לה קידודים שונים לפי שיטות המיעון. לדוגמה, פעולת ההזזה mov יכולה להתייחס

להעתקת תוכן תא זיכרון לאוגר, או להעתקת תוכן אוגר לאוגר אחר, וכן הלאה. לכל אפשרות כזאת של mov עשוי להתאים קידוד שונה.

על האסמבלר לסרוק את שורת ההוראה בשלמותה, ולהחליט לגבי הקידוד לפי האופרנדים. בדרך כלל מתחלק הקידוד לשדה של שם הפעולה, ושדות נוספים המכילים מידע לגבי שיטות המיעון. כל השדות ביחד דורשים מילה אחת או יותר בקוד המכונה.

כאשר נתקל האסמבלר בתווית המופיעה בתחילת השורה, הוא יודע שלפניו הגדרה של תווית, ואז הוא משייך לה מען – תוכנו הנוכחי של ה-IC. כך מקבלות כל התוויות את מעניהן בעת ההגדרה. תוויות אלה מוכנסות לטבלת הסמלים, המכילה בנוסף לשם התווית גם את המען ומאפיינים נוספים. כאשר תהיה התייחסות לתווית באופרנד של הוראה כלשהי, יוכל האסמבלר לשלוף את המען המתאים מטבלת הסמלים.

הוראה יכולה להתייחס גם לסמל שטרם הוגדר עד כה בתוכנית, אלא יוגדר רק בהמשך התוכנית. להלן, לדוגמה, הוראת הסתעפות למען שמוגדר על ידי התווית A שמופיעה רק בהמשך הקוד:

```
      bne A
      .
      .
      .
A:     .....
```

כאשר מגיע האסמבלר לשורת ההסתעפות (bne A), הוא טרם נתקל בהגדרת התווית A וכמובן לא יודע את המען המשוך לתווית. לכן האסמבלר לא יכול לבנות את הקידוד הבינארי של האופרנד A. נראה בהמשך כיצד נפתר בעיה זו.

בכל מקרה, תמיד אפשר לבנות במעבר הראשון את הקידוד הבינארי המלא של המילה הראשונה של כל הוראה, את הקידוד הבינארי של מילת-המידע הנוספת של אופרנד מיידי, או אוגר, וכן את הקידוד הבינארי של כל הנתונים (המתקבלים מההנחיות .data, .string).

המעבר השני

ראינו שבמעבר הראשון, האסמבלר אינו יכול לבנות את קוד המכונה של אופרנדים המשתמשים בסמלים שעדיין לא הוגדרו. רק לאחר שהאסמבלר עבר על כל התוכנית, כך שכל הסמלים נכנסו כבר לטבלת הסמלים, יכול האסמבלר להשלים את קוד המכונה של כל האופרנדים.

לשם כך מבצע האסמבלר מעבר נוסף (מעבר שני) על כל קובץ המקור, ומעדכן את קוד המכונה של האופרנדים המשתמשים בסמלים, באמצעות ערכי הסמלים מטבלת הסמלים. בסוף המעבר השני, תהיה התוכנית מתורגמת בשלמותה לקוד מכונה.

הפרדת הוראות ונתונים

בתוכנית מבחינים בשני סוגים של תוכן: הוראות ונתונים. יש לארגן את קוד המכונה כך שתהיה הפרדה בין הנתונים וההוראות. הפרדת ההוראות והנתונים לקטעים שונים בזיכרון היא שיטה עדיפה על פני הצמדה של הגדרות הנתונים להוראות המשתמשות בהן.

אחת הסכנות הטמונות באי הפרדת ההוראות מהנתונים היא, שלפעמים עלול המעבד, בעקבות שגיאה לוגית בתוכנית, לנסות "לבצע" את הנתונים כאילו היו הוראות חוקיות. למשל, שגיאה שיכולה לגרום תופעה כזו היא הסתעפות לא נכונה. התוכנית כמובן לא תעבוד נכון, אך לרוב הנוק הוא יותר חמור, כי נוצרת חריגת חומרה ברגע שהמעבד מבצע פעולה שאינה חוקית.

האסמבלר שלנו חייב להפריד, בקוד המכונה שהוא מיצר, בין קטע הנתונים לקטע ההוראות. כלומר **בקובץ הפלט (בקוד המכונה) תהיה הפרדה של הוראות ונתונים לשני קטעים נפרדים**, אם כי **בקובץ הקלט אין חובה שתהיה הפרדה כזו**. בהמשך מתואר אלגוריתם של האסמבלר, ובו פרטים כיצד לבצע את ההפרדה.

גילוי שגיאות בתוכנית המקור

כפי שהוסבר למעלה, הנחת המטלה היא שאין שגיאות בהגדרות המקור, ולכן שלב קדם האסמבלר אינו מכיל שלב גילוי שגיאות, לעומת זאת האסמבלר אמור לגלות ולדווח על שגיאות בתחביר של תוכנית המקור, כגון פעולה שאינה קיימת, מספר אופרנדים שגוי, סוג אופרנד שאינו מתאים לפעולה, שם אוגר לא קיים, ועוד שגיאות אחרות. כמו כן מוודא האסמבלר שכל סמל מוגדר פעם אחת בדיוק.

מכאן, שכל שגיאה המתגלה על ידי האסמבלר נגרמת (בדרך כלל) על ידי שורת קלט מסוימת.

לדוגמה, אם מופיעים שני אופרנדים בהוראה שאמור להיות בה רק אופרנד יחיד, האסמבלר ייתן הודעת שגיאה בנוסח "יותר מדי אופרנדים".

הערה: אם יש שגיאה בקוד האסמבלי בגוף מקרו, הרי שגיאה זו יכולה להופיע ולהתגלות שוב ושוב, בכל מקום בו נפרש המקרו. נשים לב שכאשר האסמבלר בודק שגיאות, כבר לא ניתן לזהות שזה קוד שנפרש ממקרו, כך שלא ניתן לחסוך גילויי שגיאה כפולים.

האסמבלר ידפיס את הודעות השגיאה אל הפלט הסטנדרטי stdout. בכל הודעת שגיאה יש לציין גם את מספר השורה בקובץ המקור בה זוהתה השגיאה (מניין השורות בקובץ מתחיל ב-1).

לתשומת לב: האסמבלר אינו עוצר את פעולתו אחרי שנמצאה השגיאה הראשונה, אלא ממשיך לעבור על הקלט כדי לגלות שגיאות נוספות, ככל שישנן. כמוכן שאין כל טעם לייצר את קבצי הפלט אם נתגלו שגיאות (ממילא אי אפשר להשלים את קוד המכונה).

הטבלה הבאה מפרטת מהן של שיטות המיעון החוקיות, עבור אופרנד המקור ואופרנד היעד של ההוראות השונות הקיימות בשפה הנתונה:

opcode	funct	שם ההוראה	שיטות מיעון חוקיות עבור אופרנד המקור	שיטות מיעון חוקיות עבור אופרנד היעד
0		mov	0,1,2,3	1,2,3
1		cmp	0,1,2,3	0,1,2,3
2	10	add	0,1,2,3	1,2,3
2	11	sub	0,1,2,3	1,2,3
4		lea	1,2	1,2,3
5	10	clr	אין אופרנד מקור	1,2,3
5	11	not	אין אופרנד מקור	1,2,3
5	12	inc	אין אופרנד מקור	1,2,3
5	13	dec	אין אופרנד מקור	1,2,3
9	10	jmp	אין אופרנד מקור	1,2
9	11	bne	אין אופרנד מקור	1,2
9	12	jsr	אין אופרנד מקור	1,2
12		red	אין אופרנד מקור	1,2,3
13		prn	אין אופרנד מקור	0,1,2,3
14		rts	אין אופרנד מקור	אין אופרנד יעד
15		stop	אין אופרנד מקור	אין אופרנד יעד

תהליך העבודה של האסמבלר

נתאר כעת את אופן העבודה של האסמבלר. בהמשך, יוצג אלגוריתם שלדי למעבר ראשון ושני.

האסמבלר מתחזק שני מערכים, שייקראו להלן תמונת ההוראות (code) ותמונת הנתונים (data). מערכים אלו נותנים למעשה תמונה של זיכרון המכונה (כל איבר במערך הוא בגודל מילה של המכונה, כלומר 24 סיביות). במערך ההוראות בונה האסמבלר את הקידוד של הוראות המכונה שנקראו במהלך המעבר על קובץ המקור. במערך הנתונים מכניס האסמבלר את קידוד הנתונים שנקראו מקובץ המקור (שורות הנחיה מסוג '.data' ו- '.string').

האסמבלר משתמש בשני מונים, שנקראים IC (מונה ההוראות - Instruction-Counter), ו-DC (מונה הנתונים - Data-Counter). מונים אלו מצביעים על המקום הבא הפנוי במערך ההוראות ובמערך הנתונים, בהתאמה. בכל פעם כשמתחיל האסמבלר לעבור על קובץ מקור, המונה IC מקבל ערך התחלתי 100, והמונה DC מקבל ערך התחלתי 0. הערך התחלתי IC=100 נקבע כדי שקוד המכונה של התוכנית יתאים לטעינה לזיכרון (לצורך ריצה) החל מכתובת 100.

בנוסף, מתחזק האסמבלר טבלה, אשר בה נאספות כל התוויות בהן נתקל האסמבלר במהלך המעבר על קובץ המקור. לטבלה זו קוראים טבלת-הסמלים (symbol-table). לכל סמל נשמרים בטבלה שם הסמל, ערכו המספרי, ומאפיינים נוספים (אחד או יותר), כגון המיקום בתמונת הזיכרון (data או code), וסוג הנראות של הסמל (entry או external).

במעבר הראשון האסמבלר בונה את טבלת הסמלים ואת השלד של תמונת הזיכרון (הוראות ונתונים).

האסמבלר קורא את קובץ המקור שורה אחר שורה, ופועל בהתאם לסוג השורה (הוראה, הנחיה, או שורה ריקה/הערה).

1. שורה ריקה או שורת הערה: האסמבלר מתעלם מהשורה ועובר לשורה הבאה.

2. שורת הוראה:

האסמבלר מנתח את השורה ומפענח מהי ההוראה, ומהן שיטות המיעון של האופרנדים. מספר האופרנדים נקבע בהתאם להוראה שנמצאה. שיטות המיעון נקבעות בהתאם לתחביר של כל אופרנד, כפי שהוסבר לעיל במפרט שיטות המיעון. למשל, התו # מציין מיעון מידי, תווית מציינת מיעון ישיר, שם של אוגר מציין מיעון אוגר ישיר, וכד'.

אם האסמבלר מוצא בשורת ההוראה גם הגדרה של תווית, אזי התווית (הסמל) המוגדרת מוכנסת לטבלת הסמלים. ערך הסמל בטבלה הוא IC, והמאפיין הוא code.

כעת האסמבלר קובע לכל אופרנד את ערכו באופן הבא:

- אם זה אוגר – האופרנד הוא מספר האוגר.
- אם זו תווית (מיעון ישיר) – האופרנד הוא ערך התווית כפי שמופיע בטבלת הסמלים (ייתכן והסמל טרם נמצא בטבלת הסמלים, במידה והוא יוגדר רק בהמשך התוכנית).
- אם זה התו # ואחריו מספר (מיעון מידי) – האופרנד הוא המספר עצמו.
- אם זו שיטת מיעון אחרת – ערכו של האופרנד נקבע לפי המפרט של שיטת המיעון (ראו תאור שיטות המיעון לעיל).

האסמבלר מכניס למערך ההוראות, בכניסה עליה מצביע מונה ההוראות IC, את הקוד הבינארי המלא של המילה הראשונה של ההוראה (בפורמט קידוד כפי שתואר קודם). מילה זו מכילה את קוד הפעולה וה-funct, ואת מספרי שיטות המיעון של אופרנד המקור והיעד. ה-IC מקודם ב-1.

נזכור שכאשר יש רק אופרנד אחד (כלומר אין אופרנד מקור), הסיביות של שיטת המיעון של אופרנד המקור יכילו 0. בדומה, אם זוהי הוראה ללא אופרנדים (rts, stop), אזי הסיביות של שיטות המיעון של שני האופרנדים יכילו 0.

אם זוהי הוראה עם אופרנדים (אחד או שניים), האסמבלר "משריין" מקום במערך ההוראות עבור מילות-המידע הנוספות הנדרשות בהוראה זו, ככל שנדרשות, ומקדם את IC בהתאם. כאשר אופרנד הוא בשיטת מיעון מידי או אוגר ישיר, האסמבלר מקודד גם את המילה הנוספת המתאימה במערך ההוראות. ואילו בשיטת מיעון ישיר או יחסי, מילת המידע הנוספת במערך ההוראות נשארת ללא קידוד בשלב זה.

3. שורת הנחיה:

כאשר האסמבלר קורא בקובץ המקור שורת הנחיה, הוא פועל בהתאם לסוג ההנחיה, באופן הבא:

I. 'data'.

האסמבלר קורא את רשימת המספרים, המופיעה לאחר 'data'. מכניס כל מספר אל מערך הנתונים (בקידוד בינארי), ומקדם את מצביע הנתונים DC ב-1 עבור כל מספר שהוכנס.

אם בשורה 'data' מוגדרת גם תווית, אזי התווית מוכנסת לטבלת הסמלים. ערך התווית הוא ערך מונה הנתונים DC שלפני הכנסת המספרים למערך. המאפיין של התווית הוא 'data'.

II. 'string'.

הטיפול ב-'string' דומה ל-'data', אלא שקודי ה-ascii של התווים הם אלו המוכנסים אל מערך הנתונים (כל תו במילה נפרדת). לבסוף מוכנס למערך הנתונים הערך 0 (המציין סוף מחרוזת). המונה DC מקודם באורך המחרוזת + 1 (גם התו המסיים את המחרוזת תופס מקום).

הטיפול בתווית המוגדרת בהנחיה 'string' זהה לטיפול הנעשה בהנחיה 'data'.

III. 'entry'.

זוהי הנחיה לאסמבלר לאפיין את התווית הנתונה כאופרנד כ-`entry` בטבלת הסמלים. בעת הפקת קבצי הפלט (ראו בהמשך), התווית תירשם בקובץ ה-`entries`.
לתשומת לב: זה לא נחשב כשגיאה אם בקובץ המקור מופיעה יותר מהנחיית `entry`. אחת עם אותה תווית כאופרנד. המופעים הנוספים אינם מוסיפים דבר, אך גם אינם מפריעים.

IV. 'extern'.

זוהי הצהרה על סמל (תווית) המוגדר בקובץ מקור אחר, והקובץ הנוכחי עושה בו שימוש. האסמבלר מכניס את הסמל המופיע כאופרנד לטבלת הסמלים, עם הערך 0 (הערך האמיתי לא ידוע, וייקבע רק בשלב הקישור), ועם המאפיין `external`. לא ידוע באיזה קובץ נמצאת הגדרת הסמל, ואין זה רלוונטי עבור האסמבלר.
לתשומת לב: זה לא נחשב כשגיאה אם בקובץ המקור מופיעה יותר מהנחיית `extern`. אחת עם אותה תווית כאופרנד. המופעים הנוספים אינם מוסיפים דבר, אך גם אינם מפריעים.
לתשומת לב: באופרנד של הוראה או של הנחית `entry`, מותר להשתמש בסמל אשר יוגדר בהמשך הקובץ (אם באופן ישיר על ידי הגדרת תווית, ואם באופן עקיף על ידי הנחית `extern`).

בסוף המעבר הראשון, האסמבלר מעדכן בטבלת הסמלים כל סמל המאופיין כ-`data`, על ידי הוספת $IC + 100$ (עשרוני) לערכו של הסמל. הסיבה לכך היא שבתמונה הכוללת של קוד המכונה, תמונת הנתונים מופרדת מתמונת הוראות, וכל הנתונים נדרשים להופיע בקוד המכונה אחרי כל ההוראות. סמל מסוג `data` הוא תווית בתמונת הנתונים, והעדכון מוסיף לערך הסמל (כלומר לכתובתו בזיכרון) את האורך הכולל של תמונת ההוראות, בתוספת כתובת התחלת הטעינה של הקוד, שהיא 100.

טבלת הסמלים מכילה כעת את ערכי כל הסמלים הנחוצים להשלמת תמונת הזיכרון (למעט ערכים של סמלים חיצוניים).

במעבר השני, האסמבלר משלים באמצעות טבלת הסמלים את קידוד כל המילים במערך ההוראות שטרם קודדו במעבר הראשון. במודל המכונה שלנו אלו הן מילות-מידע נוספות של הוראות, אשר מקודדות אופרנד בשיטת מיעון ישיר או יחסי.

אלגוריתם שלדי של האסמבלר

לחידוד ההבנה של תהליך העבודה של האסמבלר, נציג להלן אלגוריתם שלדי למעבר הראשון ולמעבר השני.

לתשומת לב: אין חובה להשתמש דווקא באלגוריתם זה.

כאמור, אנו מחלקים את תמונת קוד המכונה לשני חלקים: תמונת ההוראות (`code`), ותמונת הנתונים (`data`). לכל חלק נתחזק מונה נפרד: `IC` (מונה ההוראות) ו-`DC` (מונה הנתונים).

נבנה את קוד המכונה כך שיתאים לטעינה לזיכרון החל מכתובת 100.

בכל מעבר מתחילים לקרוא את קובץ המקור מהתחלה.

מעבר ראשון

1. אתחל $DC \leftarrow 0, IC \leftarrow 100$.
2. קרא את השורה הבאה מקובץ המקור. אם נגמר קובץ המקור, עבור ל-17.
3. האם השדה הראשון בשורה הוא תווית? אם לא, עבור ל-5.
4. הדלק דגל "יש הגדרת סמל".
5. האם זוהי הנחיה לאחסון נתונים, כלומר, האם הנחית data או string? אם לא, עבור ל-8.
6. אם יש הגדרת סמל (תווית), הכנס אותו לטבלת הסמלים עם המאפיין data. ערך הסמל יהיה בסיס והיסט. (אם הסמל אינו תווית חוקית, או שהסמל כבר נמצא בטבלה, יש להודיע על שגיאה).
7. זהה את סוג הנתונים, קודד אותם בתמונת הנתונים, והגדל את מונה הנתונים DC על ידי הוספת האורך הכולל של הנתונים שהוגדרו בשורה הנוכחית. חזור ל-2.
8. האם זו הנחית extern או הנחית entry? אם לא, עבור ל-11.
9. אם זוהי הנחית entry. חזור ל-2 (ההנחיה תטופל במעבר השני).
10. אם זו הנחית extern, הכנס את הסמל המופיע כאופרנד של ההנחיה לתוך טבלת הסמלים עם פעמיים הערך 0 (בסיס והיסט), ועם המאפיין external. (אם הסמל אינו תווית חוקית, או שהסמל כבר נמצא בטבלה ללא המאפיין external, יש להודיע על שגיאה). חזור ל-2.
11. זוהי שורת הוראה. אם יש הגדרת סמל (תווית), הכנס אותו לטבלת הסמלים עם המאפיין code. ערכו של הסמל יהיה בסיס והיסט (אם הסמל אינו תווית חוקית, או שהסמל כבר נמצא בטבלה, יש להודיע על שגיאה).
12. חפש את שם הפעולה בטבלת שמות הפעולות, ואם לא נמצא, אז הודע על שגיאה בשם ההוראה.
13. נח את מבנה האופרנדים של ההוראה, וחשב מהו מספר המילים הכולל שתופסת ההוראה בקוד המכונה (נקרא למספר זה L).
14. בנה כעת את הקוד הבינארי של המילה הראשונה של ההוראה, ושל כל מילת-מידע נוספת המקודדת אופרנד במיעון מידי. אפשר לקודד גם את המילה השנייה בקוד ההוראה (אם קיימת).
15. שמור את הערכים IC ו-L יחד עם נתוני קוד המכונה של ההוראה.
16. עדכן $IC \leftarrow IC + L$, וחזור ל-2.
17. קובץ המקור נקרא בשלמותו. אם נמצאו שגיאות במעבר הראשון, עצור כאן.
18. שמור את הערכים הסופיים של IC ושל DC (נקרא להם ICF ו-DCF). נשתמש בהם לבניית קבצי הפלט, אחרי המעבר השני.
19. עדכן בטבלת הסמלים את ערכו של כל סמל המאופיין כ-data, ע"י שימוש בערך ICF באופן הבא: ראשית יש לחשב את ערכו המלא המעודכן של הסמל בהינתן ICF-והבסיס+היסט הנוכחיים (ככל שזה מופיע כך בטבלה), ואז לחשב בסיס+היסט חדשים.
20. התחל מעבר שני.

מעבר שני

1. קרא את השורה הבאה מקובץ המקור. אם נגמר קובץ המקור, עבור ל-7.
2. אם השדה הראשון בשורה הוא סמל (תווית), דלג עליו.
3. האם זוהי הנחית data או string או extern? אם כן, חזור ל-1.
4. האם זוהי הנחית entry? אם לא, עבור ל-6.
5. הוסף בטבלת הסמלים את המאפיין entry למאפייני הסמל המופיע כאופרנד של ההנחיה (אם הסמל לא נמצא בטבלת הסמלים, יש להודיע על שגיאה). חזור ל-1.
6. השלם את הקידוד הבינארי של מילות-המידע של האופרנדים, בהתאם לשיטות המיעון שבשימוש. לכל אופרנד בקוד המקור המכיל סמל, מצא את ערכו של הסמל בטבלת הסמלים (אם הסמל לא נמצא בטבלה, יש להודיע על שגיאה). אם הסמל מאופיין external, הוסף את כתובת מילת-המידע הרלוונטית לרשימת מילות-מידע שמתייחסות לסמל חיצוני. לפי הצורך, לחישוב הקידוד והכתובות, אפשר להיעזר בערכים IC ו-L של ההוראה, כפי שנשמרו במעבר

- הראשון. חזור ל- 1. לתשומת לב: יש להשלים שתי מילות מידע לכל אופרנד. כמו כן, אם מדובר בסמל חיצוני, יש לרשום בקובץ ext את הכתובות של שתי מילות המידע. לפי הגדרת השפה, כתובת מילת ההיסט תהיה תמיד עוקבת לכתובת מילת הבסיס (דוגמת קובץ ext בהמשך).
7. קובץ המקור נקרא בשלמותו. אם נמצאו שגיאות במעבר השני, עצור כאן.
8. בנה את קבצי הפלט (פרטים נוספים בהמשך).

נפעיל אלגוריתם זה על תוכנית הדוגמה שראינו למעלה (לאחר שלב פרישת המקרואים), ונציג את הקוד הבינארי שמתקבל במעבר ראשון ובמעבר שני. להלן שוב תכנית הדוגמה.

```
; file ps.as
.entry LIST
.extern W
MAIN:      add    r3, LIST
LOOP:      prn    #48
           lea    STR, r6
           inc    r6
           mov    r3, W
           sub    r1, r4
           bne    END
           cmp    val1, #-6
           bne    END[r15]
           dec    K

.entry MAIN
           sub     LOOP[r10], r14
END:      stop
STR:      .string "abcd"
LIST:     .data  6, -9
           .data  -100

.entry K
K:        .data  31
.extern val1
```

נבצע מעבר ראשון על הקוד לעיל, ונבנה את טבלת הסמלים. כמו כן, נשלים במעבר זה את הקידוד של כל תמונת הנתונים, ושל המילה הראשונה של כל הוראה (נשים לב שיש לקודד גם את המילה השנייה). כמו כן, נקודד מילות-מידע נוספות של כל הוראה, ככל שקידוד זה אינו תלוי בערך של סמל. את מילות-המידע שעדיין לא ניתן לקודד במעבר הראשון נסמן ב "?" בדוגמה להלן.

Address (decimal)	Source Code	Machine Code (binary)																		
		19									9	8	7	6	5	4	3	2	1	0
0100	MAIN: add r3, LIST	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0101		0	1	0	0	1	0	1	0	0	0	1	1	1	1	0	0	0	0	1
0102		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0103		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0104	LOOP: prn #48	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0105		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0106		0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
0107	lea STR, r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0108		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1
0109		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0110		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0111	inc r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0112		0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	1
0113	mov r3, W	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0114		0	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	1
0115		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0116		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0117	sub r1, r4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0118		0	1	0	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	1
0119	bne END	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0120		0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	1

Address (decimal)	Source Code	Machine Code (binary)																		
		19	-	-	-	-	-	-	-	-	9	8	7	6	5	4	3	2	1	0
0121	cmp val1, #-6	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0122		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0123		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0124		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0125		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0126		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
0127		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
0128		bne END[r15]	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0129	0		1	0	0	1	0	1	1	0	0	0	0	0	0	1	1	1	1	0
0130	?		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0131	?		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0132	dec K	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	
0133		0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	1	
0134		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0135		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0136	sub LOOP[r10],r14	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0137		0	1	0	0	1	0	1	1	1	0	1	0	1	0	1	1	1	0	1
0138		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0139		?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	
0140	END: stop	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	
0141	STR: .string "abcd"	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0142		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0143		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0144		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0
0145		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0146	LIST: .data 6, -9	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0147		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
0148	.data -100	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	0
0149	K: .data 31	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

טבלת הסמלים אחרי מעבר ראשון היא :

Symbol	Value (decimal)	Base address	offset	Attributes
W	0	0	0	external
MAIN	100	96	4	code, entry
LOOP	104	96	8	code
END	140	128	12	code
STR	141	128	13	data
LIST	146	144	2	data, entry
K	149	144	5	data, entry
val1	0	0	0	external

נבצע עתה את המעבר השני. נשלים באמצעות טבלת הסמלים את הקידוד החסר במילים המסומנות "?". הקוד הבינארי בצורתו הסופית כאן זהה לקוד שהוצג בתחילת הנושא "אסמבלר עם שני מעברים".

הערה : כאמור, האסמבלר בונה קוד מכונה כך שיתאים לטעינה לזיכרון החל מכתובת 100 (עשרוני). אם הטעינה בפועל (לצורך הרצת התוכנית) תהיה לכתובת אחרת, יידרשו תיקונים בקוד הבינארי בשלב הטעינה, שיוכנסו בעזרת מידע נוסף שהאסמבלר מכין בקבצי הפלט (ראו בהמשך).

Address (decimal)	Source Code	Machine Code (binary)																			
		19									9	8	7	6	5	4	3	2	1	0	
0100	MAIN: add r3, LIST	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	
0101		0	1	0	0	1	0	1	0	0	0	1	1	1	1	0	0	0	0	1	
0102		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	
0103		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	
0104	LOOP: prn #48	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	
0105		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0106		0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	
0107	lea STR, r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	
0108		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1	
0109		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	
0110		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
0111	inc r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	
0112		0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	1	1
0113	mov r3, W	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	
0114		0	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	1	
0115		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0116		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0117	sub r1, r4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	
0118		0	1	0	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	1	1
0119	bne END	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0120		0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	
0121		0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	
0122		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0123	cmp val1, #-6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	
0124		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	
0125		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0126		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0127		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
0128	bne END[r15]	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0129		0	1	0	0	1	0	1	1	0	0	0	0	0	0	1	1	1	1	0	
0130		0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	
0131		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0132	dec K	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	
0133		0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	1	
0134		0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	
0135		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
0136	sub LOOP[r10], r14	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0137		0	1	0	0	1	0	1	1	1	0	1	0	1	0	1	1	1	0	1	1
0138		0	0	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0
0139		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0140	END: stop	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0141	STR: .string "abcd"	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	1
0142		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	0
0143		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	1
0144		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0	0
0145		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0146	LIST: .data 6, -9	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0147		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
0148	.data -100	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	0	0
0149	K: .data 31	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

טבלת הסמלים אחרי מעבר שני היא :

Symbol	Value (decimal)	Base address	offset	Attributes
W	0	0	0	external
MAIN	100	96	4	code, entry
LOOP	104	96	8	code
END	140	128	12	code
STR	141	128	13	data
LIST	146	144	2	data, entry
K	149	144	5	data, entry
val1	0	0	0	external

בסוף המעבר השני, אם לא נתגלו שגיאות, האסמבלר בונה את קבצי הפלט (ראו בהמשך), שמכילים את הקוד הבינארי ומידע נוסף עבור שלבי הקישור והטעינה. כאמור, שלבי הקישור והטעינה אינם למימוש בפרויקט זה, ולא נדון בהם כאן.

קבצי קלט ופלט של האסמבלר

בהפעלה של האסמבלר, יש להעביר אליו באמצעות ארגומנטים של שורת הפקודה (command line arguments) רשימה של שמות קבצי מקור (אחד או יותר). אלו הם קבצי טקסט, ובהם תוכניות בתחביר של שפת האסמבלי שהוגדרה בממ"ן זה.

האסמבלר פועל על כל קובץ מקור בנפרד, ויוצר עבורו קבצי פלט כדלקמן :

- קובץ `.am`, המכיל את קובץ המקור לאחר שלב קדם האסמבלר (לאחר פרישת המקרואים)
- קובץ `object`, המכיל את קוד המכונה.
- קובץ `externals`, ובו פרטים על כל המקומות (הכתובות) בקוד המכונה בהם יש מילת-מידע שמקודדת ערך של סמל שהוצהר כחיצוני (סמל שהופיע כאופרנד של הנחיית `extern`, ומאופיין בטבלת הסמלים כ- `external`).
- קובץ `entries`, ובו פרטים על כל סמל שמוצהר כנקודת כניסה (סמל שהופיע כאופרנד של הנחיית `entry`, ומאופיין בטבלת הסמלים כ- `entry`).

אם אין בקובץ המקור אף הנחיית `extern`, האסמבלר לא יוצר את קובץ הפלט מסוג `externals`.
אם אין בקובץ המקור אף הנחיית `entry`, האסמבלר לא יוצר את קובץ הפלט מסוג `entries`.

שמות קבצי המקור חייבים להיות עם הסיומת `“.as”`. למשל, השמות `x.as`, `y.as`, ו-`hello.as` הם שמות חוקיים. העברת שמות הקבצים הללו כארגומנטים לאסמבלר נעשית ללא ציון הסיומת.

לדוגמה : נניח שתוכנית האסמבלר שלנו נקראת `assembler`, אזי שורת הפקודה הבאה :

```
assembler x y hello
```

תריץ את האסמבלר על הקבצים : `x.as`, `y.as`, `hello.as`.

שמות קבצי הפלט מבוססים על שם קובץ הקלט, כפי שהופיע בשורת הפקודה, בתוספת סיומת מתאימה : הסיומת `“.am”` עבור קובץ לאחר פרישת מאקרו, הסיומת `“.ob”` עבור קובץ ה-`object`, הסיומת `“.ent”` עבור קובץ ה-`entries`, והסיומת `“.ext”` עבור קובץ ה-`externals`.

לדוגמה, בהפעלת האסמבלר באמצעות שורת הפקודה : `assembler x`
יווצר קובץ פלט `x.ob`, וכן קבצי פלט `x.ent` ו-`x.ext` ככל שיש הנחיות `entry` או `extern`. בקובץ המקור, אם אין מאקרו בקובץ המקור, אזי קובץ `“.am”` יהיה זהה לקובץ `“.as”`.

נציג כעת את הפורמטים של קבצי הפלט. דוגמאות יובאו בהמשך.

פורמט קובץ ה-object

קובץ זה מכיל את תמונת הזיכרון של קוד המכונה, בשני חלקים: תמונת ההוראות ראשונה, ואחריה ובצמוד תמונת הנתונים.

כזכור, האסמבלר מקודד את ההוראות כך שתמונת ההוראות תתאים לטעינה החל מכתובת 100 (עשרוני) בזיכרון. נשים לב שרק בסוף המעבר הראשון יודעים מהו הגודל הכולל של תמונת ההוראות. מכיוון שתמונת הנתונים נמצאת אחרי תמונת ההוראות, גודל תמונת ההוראות משפיע על הכתובות בתמונת הנתונים. זו הסיבה שבגללה היה צורך לעדכן בטבלת הסמלים, בסוף המעבר הראשון, את ערכי הסמלים המאופיינים כ-data (כזכור, באלגוריתם השלדי שהוצג לעיל, בצעד 19, הוספנו לכל סמל כזה את הערך ICF). במעבר השני, בהשלמת הקידוד של מילות-המידע, משתמשים בערכים המעודכנים של הסמלים, המותאמים למבנה המלא והסופי של תמונת הזיכרון.

כעת האסמבלר יכול לכתוב את תמונת הזיכרון בשלמותה לתוך קובץ פלט (קובץ ה-object).

השורה הראשונה בקובץ ה-object היא "כותרת", המכילה שני מספרים (בבסיס עשרוני): הראשון הוא האורך הכולל של תמונת ההוראות (במילות זיכרון), והשני הוא האורך הכולל של תמונת הנתונים (במילות זיכרון). בין שני המספרים מפריד רווח אחד. כזכור, במעבר הראשון, בצעד 19, נשמרו ערכי הסמלים תוך שימוש ב-ICF.

השורות הבאות בקובץ מכילות את תמונת הזיכרון. בכל שורה זוג שדות: כתובת של מילה בזיכרון, ותוכן המילה. הכתובת תירשם בבסיס עשרוני בארבע ספרות (כולל אפסים מובילים). תוכן המילה יירשם **בבסיס "מיוחד"** ב-3 ספרות (כולל אפסים מובילים). בין השדות בשורה יש רווח אחד.

"בסיס מיוחד"

כל שורה בתמונת הזיכרון היא באורך 20 סיביות, החל מסיבית 0 (מימין) ועד לסיבית 19 (משמאל). נחלק את 20 הסיביות ל-5 קבוצות בנות 4 סיביות בכל קבוצה כך:

- A סיביות 16-19 יקראו קבוצה A
- B סיביות 12-15 יקראו קבוצה B
- C סיביות 8-11 יקראו קבוצה C
- D סיביות 4-7 יקראו קבוצה D
- E סיביות 0-3 יקראו קבוצה E

כל 4 סיביות של קבוצה מסוימת יומרו לספרה הקסאדצימלית וייכתבו לקובץ ה-OB באופן הבא: תחילה ייכתב שם הקבוצה באות גדולה, ולאחריה הייצוג ההקסאדצימלי של סיביות הקבוצה. כך ייכתבו כל הסיביות מכל הקבוצות עם הפרדה של תו מקף (-) בין קבוצה לקבוצה. לדוגמה, נסתכל על 20 הסיביות הבאות:

0	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

הם ייכתבו לקובץ ה-OB כך:

A4-B0-C3-De-E1

פורמט קובץ ה-entries

קובץ ה-entries בנוי משורות טקסט, שורה אחת לכל סמל שמאופיין בטבלת הסמלים כ-entry. בשורה מופיע שם הסמל, ולאחריה כתובת הבסיס שלו וההיסט, כפי שנקבע בטבלת הסמלים (בבסיס עשרוני בארבע ספרות, כולל אפסים מובילים). אין חשיבות לסדר השורות, כי כל שורה עומדת בפני עצמה. בין השדות בשורה יש פסיק אחד.

פורמט קובץ ה-externals

קובץ ה-externals בנוי אף הוא משורות טקסט, שורה לכל כתובת בקוד המכונה בה יש מילת מידע המתייחסת לסמל שמאופיין כ-external. כזכור, רשימה של מילות-מידע אלה נבנתה במעבר השני (צעד 6 באלגוריתם השלדי).

כל שורה בקובץ ה-externals מכילה את שם הסמל החיצוני, ולאחריו המילה BASE ולאחריה הכתובת של מילת-המידע בקוד המכונה בה נדרשת כתובת הבסיס (בבסיס עשרוני בארבע ספרות, כולל אפסים מובילים). ולאחר מכן, שורה נוספת המכילה את שם הסמל החיצוני, ולאחריו המילה OFFSET ולאחריה הכתובת של מילת-המידע בקוד המכונה בה נדרש ההיסט (OFFSET) (בבסיס עשרוני בארבע ספרות, כולל אפסים מובילים). בין השדות בשורה יש רווח אחד. אין חשיבות לסדר השורות, כי כל שורה עומדת בפני עצמה.

לתשומת לב: ייתכן ויש מספר כתובות בקוד המכונה בהן מילות-המידע מתייחסות לאותו סמל חיצוני. לכל כתובת כזו תהיה שורה נפרדת בקובץ ה-externals.

נדגים את הפלט שמייצר האסמבלר עבור קובץ מקור בשם ps.as שהודגם קודם לכן.

התוכנית לאחר שלב פרישת המקרו תיראה כך:

```
; file ps.am
.entry LIST
.extern W

MAIN:      add    r3, LIST
LOOP:      prn    #48
           lea    STR, r6
           inc    r6
           mov    r3, W
           sub    r1, r4
           bne    END
           cmp    val1, #-6
           bne    END[r15]
           dec    K

.entry MAIN
           sub    LOOP[r10], r14
END:       stop
STR:       .string "abcd"
LIST:      .data  6, -9
           .data  -100

.entry K
K:         .data  31
.extern val1
```


להלן הקידוד הבינארי המלא (תמונת הזיכרון) של קובץ המקור, בגמר המעבר השני.

Address (decimal)	Source Code	Machine Code (binary)																		
		19	-	-	-	-	-	-	-	-	9	8	7	6	5	4	3	2	1	0
0100	MAIN: add r3, LIST	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0101		0	1	0	0	1	0	1	0	0	0	1	1	1	1	0	0	0	0	1
0102		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0
0103		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0104	LOOP: prn #48	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0105		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0106		0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
0107	lea STR, r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0108		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1
0109		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0110		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0111	inc r6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0112		0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	0	1
0113	mov r3, W	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0114		0	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	1
0115		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0116		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0117	sub r1, r4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0118		0	1	0	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	1
0119	bne END	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0120		0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	1
0121		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0122		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0123	cmp val1, #-6	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0124		0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0125		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0126		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0127		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
0128	bne END[r15]	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0129		0	1	0	0	1	0	1	1	0	0	0	0	0	0	1	1	1	1	0
0130		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0131		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0
0132	dec K	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0133		0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	1
0134		0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0
0135		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0136	sub LOOP[r10], r14	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0137		0	1	0	0	1	0	1	1	1	0	1	0	1	0	1	1	1	0	1
0138		0	0	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
0139		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0140	END: stop	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0141	STR: .string "abcd"	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0142		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0143		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
0144		0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0
0145		0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0146	LIST: .data 6, -9	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0147		0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
0148	.data -100	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	0
0149	K: .data 31	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

טבלת הסמלים בגמר המעבר השני היא :

Symbol	Value (decimal)	Base address	offset	Attributes
W	0	0	0	external
MAIN	100	96	4	code, entry
LOOP	104	96	8	code
END	140	128	12	code
STR	141	128	13	data
LIST	146	144	2	data, entry
K	149	144	5	data, entry
val1	0	0	0	external

להלן תוכן קבצי הפלט של הדוגמה.

הקובץ ps.ob:

41 9

0100 A4-B0-C0-D0-E4
0101 A4-Ba-C3-Dc-E1
0102 A2-B0-C0-D9-E0
0103 A2-B0-C0-D0-E2
0104 A4-B2-C0-D0-E0
0105 A4-B0-C0-D0-E0
0106 A4-B0-C0-D3-E0
0107 A4-B0-C0-D1-E0
0108 A4-B0-C0-D5-Eb
0109 A2-B0-C0-D8-E0
0110 A2-B0-C0-D0-Ed
0111 A4-B0-C0-D2-E0
0112 A4-Bc-C0-D1-Eb
0113 A4-B0-C0-D0-E1
0114 A4-B0-C3-Dc-E1
0115 A1-B0-C0-D0-E0
0116 A1-B0-C0-D0-E0
0117 A4-B0-C0-D0-E4
0118 A4-Bb-C1-Dd-E3
0119 A4-B0-C2-D0-E0
0120 A4-Bb-C0-D0-E1
0121 A2-B0-C0-D8-E0
0122 A2-B0-C0-D0-Ec
0123 A4-B0-C0-D0-E2
0124 A4-B0-C0-D4-E0
0125 A1-B0-C0-D0-E0
0126 A1-B0-C0-D0-E0
0127 A4-Bf-Cf-Df-Ea
0128 A4-B0-C2-D0-E0
0129 A4-Bb-C0-D3-Ee

0130 A2-B0-C0-D8-E0
 0131 A2-B0-C0-D0-Ec
 0132 A4-B0-C0-D2-E0
 0133 A4-Bd-C0-D0-E1
 0134 A2-B0-C0-D9-E0
 0135 A2-B0-C0-D0-E5
 0136 A4-B0-C0-D0-E4
 0137 A4-Bb-Ca-Db-Eb
 0138 A2-B0-C0-D6-E0
 0139 A2-B0-C0-D0-E8
 0140 A4-B8-C0-D0-E0
 0141 A4-B0-C0-D6-E1
 0142 A4-B0-C0-D6-E2
 0143 A4-B0-C0-D6-E3
 0144 A4-B0-C0-D6-E4
 0145 A4-B0-C0-D0-E0
 0146 A4-B0-C0-D0-E6
 0147 A4-Bf-Cf-Df-E7
 0148 A4-Bf-Cf-D9-Ec
 0149 A4-B0-C0-D1-Ef

הסובץ ps.ent :

MAIN,96,4
 LIST,144,2
 K,144,5

הסובץ ps.ext :

W BASE 115
 W OFFSET 116

 vall BASE 125
 vall OFFSET 126

סיכום והנחיות כלליות

- גודל תוכנית המקור הניתנת כקלט לאסמבלר אינו ידוע מראש, ולכן גם גודלו של קוד המכונה אינו צפוי מראש. אולם בכדי להקל במימוש האסמבלר, מותר להניח גודל מקסימלי. לפיכך יש אפשרות להשתמש במערכים לאחסון תמונת קוד המכונה בלבד. כל מבנה נתונים אחר (למשל טבלת הסמלים וטבלת המקרו), יש לממש באופן יעיל וחסכוני (למשל באמצעות רשימה מקושרת והקצאת זיכרון דינאמי).
- השמות של קבצי הפלט צריכים להיות תואמים לשם קובץ הקלט, למעט הסיומות. למשל, אם קובץ הקלט הוא prog.as אזי קבצי הפלט שיווצרו הם : prog.ob, prog.ext, prog.ent
- מתכונת הפעלת האסמבלר צריכה להיות כפי הנדרש בממ"ן, ללא שינויים כלשהם. כלומר, ממשיק המשתמש יהיה אך ורק באמצעות שורת הפקודה. בפרט, שמות קבצי המקור יועברו לתוכנית האסמבלר כארגומנטים (אחד או יותר) בשורת הפקודה. אין להוסיף תפריטי קלט אינטראקטיביים, חלונות גרפיים למיניהם, וכד'.
- יש להקפיד לחלק את מימוש האסמבלר למספר מודולים (קבצים בשפת C) לפי משימות. אין לרכז משימות מסוגים שונים במודול יחיד. מומלץ לחלק למודולים כגון : מעבר ראשון, מעבר שני, פונקציות עזר (למשל, תרגום לבסיס, ניתוח תחבירי של שורה), טבלת הסמלים, מפת הזיכרון, טבלאות קבועות (קודי הפעולה, שיטות המיעון החוקיות לכל פעולה, וכד').
- יש להקפיד ולתעד את המימוש באופן מלא וברור, באמצעות הערות מפורטות בקוד.
- יש לאפשר תווים לבנים עודפים בקובץ הקלט בשפת אסמבלי. למשל, אם בשורת הוראה יש שני אופרנדים המופרדים בפסיק, אזי לפני ואחרי הפסיק מותר שיהיו רווחים וטאבים בכל כמות. בדומה, גם לפני ואחרי שם הפעולה. מותרות גם שורות ריקות. האסמבלר יתעלם מתווים לבנים מיותרים (כלומר ידלג עליהם).
- הקלט (קוד האסמבלי) עלול להכיל שגיאות תחביריות. על האסמבלר לגלות ולדווח על כל השגות השגויות בקלט. אין לעצור את הטיפול בקובץ קלט לאחר גילוי השגיאה הראשונה. יש להדפיס למסך הודעות מפורטות ככל הניתן, כדי שאפשר יהיה להבין מה והיכן כל שגיאה. כמובן שאם קובץ קלט מכיל שגיאות, אין טעם להפיק עבורו את קבצי הפלט (ob, ext, ent).

תם ונשלם פרק ההסברים והגדרת הפרויקט.

בהצלחה!